

Advanced Lesson 10 - Exercises:

Modules, Recursion and Procedural Programming

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: A Module that averages ten integers

```
Clear[m1];  
m1 := Module[{list}, list = Table[i, {i, 1, 10}]; Mean[list]]
```

m1

Module's first argument is the list of local names. Everything inside is evaluated with those names shadowed by private copies, so the symbol "list" used here cannot collide with anything of the same name outside. The last expression in the body is the return value - there is no return keyword and none is wanted.

The average of 1 through 10 is $11/2$, returned exactly rather than as 5.5, because the inputs were exact integers.

2: A Module that plots ten ordered pairs

```
Clear[m2];  
m2 := Module[{pairs}, pairs = Table[{i, i^2}, {i, 1, 10}];  
ListPlot[pairs, PlotStyle -> PointSize[0.02`]]
```

m2

The ten points {1, 1} through {10, 100} lie on a parabola. The module returns the graphic itself, which is an ordinary expression like any other and can be assigned, combined with Show, or handed to further processing.

3: A Module wrapping DSolve and Plot

```
Clear[f, a, y0, t, y];
f[a_, y0_] := Module[{sol, yt}, sol = DSolve[{y'[t] == a y[t], y[0] == y0}, y[t], t];
  yt = y[t] /. First[sol];
  Plot[yt, {t, 0, 2}, PlotRange -> All, PlotLabel -> "a = "a", y0 = "y0"]]

f[2, 3]
f[-2, 3]
```

The exercise writes this with immediate assignment, `f[a_, y0_] = Module[...]`. Delayed assignment is used here instead, and deliberately: with `"="` the Module body would be evaluated once at definition time, with `a` and `y0` still bare symbols, and `DSolve` would be asked to solve a problem whose parameters do not exist yet. Everything in this body depends on the arguments, so it all has to be postponed. This is the Lesson 2 lesson again, in a bigger setting.

The solutions are $y[t] = 3 E^{(2t)}$ and $y[t] = 3 E^{(-2t)}$: exponential growth for positive `a`, decay for negative `a`, both starting at 3. `PlotRange -> All` keeps the whole growing curve in frame rather than clipping it.

4: The same Module with y0 localised and initialised to 1

```
Clear[f, a, t, y];
f[a_] := Module[{y0 = 1, sol, yt}, sol = DSolve[{y'[t] == a y[t], y[0] == y0}, y[t], t];
  yt = y[t] /. First[sol];
  Plot[yt, {t, 0, 2}, PlotRange -> All, PlotLabel -> "a = "a", y0 = "y0"]]

f[2]
f[-2]
```

A local variable can be given an initial value inside the Module's variable list, using ordinary assignment syntax there. `y0` is now private to the module and fixed at 1, so it has vanished from the function's signature. The curves are $E^{(2t)}$ and $E^{(-2t)}$, both starting at 1.

5: A Module that averages the first n integers

```
Clear[f, n];
f[n_] := Module[{list}, list = Table[i, {i, 1, n}]; Mean[list]]
{f[10], f[100]}
```

f[10] is 11/2 and f[100] is 101/2 - exact rationals, and both instances of the general formula $(n + 1)/2$. Generalising Exercise 1 cost exactly one pattern variable.

6: A linear economy, defined recursively

```
Clear[economy, a, b, n, value1];
economy[1] = value1;
economy[n_] := a economy[n - 1] + b
{economy[1], economy[2], economy[10]}
```

The base case is stored as a literal definition and the recursive case as a pattern definition, and Mathematica's ordering puts the literal first, so the recursion terminates without any explicit test.

economy[1] is value1. economy[2] is b + a value1. economy[10] is a deeply nested expression, which Mathematica leaves in nested form rather than expanding - apply Expand or Simplify to see it as a polynomial in a with the geometric-series structure made explicit.

7: A nonlinear economy, defined through an arbitrary F

```
Clear[economy, F, X, a, b, n, value1];
F[X_] := a X + b;
economy[1] = value1;
economy[n_] := F[economy[n - 1]]
{economy[1], economy[2], economy[10]}
```

The recursion is now written in terms of an unspecified transition function F, with the linear choice supplied separately. The values agree with Exercise 6 exactly, which is the point - the structure of the model and the particular law of motion have been separated, and replacing F with something genuinely nonlinear would need no change to the recursion at all.

8: A two-lag economy

```
Clear[economy, F, X, Y, a, b, n, value1, value2];
F[X_, Y_] := a X + b Y;
economy[1] = value1;
economy[2] = value2;
economy[n_] := F[economy[n - 1], economy[n - 2]]

{economy[1], economy[2], economy[3], economy[5]}
```

Two lags need two base cases, or the recursion runs off the bottom. This is the Fibonacci shape, and it has Fibonacci's cost: `economy[n]` calls itself twice, so the number of calls grows exponentially and `economy[30]` would be unpleasant. The standard cure is to make the definition remember its own results - writing the recursive case as `economy[n_] := economy[n] = ...` so that each computed value is stored as a new literal definition. That is the memoisation idiom, and it turns exponential work into linear work.

9: The same economy, procedurally - Do, While and For

```
Clear[economyDo, economyWhile, economyFor, a, b, n, k, value1];
economyDo[n_] := Module[{e = value1}, Do[e = a e + b, {k, 2, n}];
  e]

economyWhile[n_] := Module[{e = value1, k = 1}, While[k < n, e = a e + b; k = k + 1]; e]

economyFor[n_] := Module[{e = value1, k}, For[k = 2, k ≤ n, k = k + 1, e = a e + b]; e]

{economyDo[1], economyDo[2], economyDo[10]}

{economyWhile[1], economyWhile[2], economyWhile[10]}

{economyFor[1], economyFor[2], economyFor[10]}
```

All three loops produce the same values as the recursion in Exercise 6, and each returns `value1` unchanged when `n` is 1, because none of them executes its body at all in that case.

The three constructs differ only in where the bookkeeping lives. `Do` takes an iterator specification and manages the counter itself. `While` takes a condition and requires the body to advance the counter, or it never terminates. `For` takes initialisation, test, increment and body as four separate arguments, in the C order. `Do` is the one to reach for when the number of repetitions is known in advance, which it is here; the other two are shown for the comparison the exercise asks for.

Note that the counter is declared local in each case. A loop variable that

leaks into the global context is one of the standard ways a notebook starts producing results that depend on what was evaluated before it.

10: A four-piece function, via Which

```
Clear[f, x];
f[x_] := Which[x < -2, x^2, -2 ≤ x < 1, Sin[x^2], 1 ≤ x < 3, 2 x, 3 ≤ x, x]
Plot[f[x], {x, -3, 4}]
```

Which evaluates its tests in order and returns the branch belonging to the first True, so the four intervals are checked left to right and the half-open boundaries fall out of the ordering without any special handling. The plot shows a parabola arriving from the left down to 4 at $x = -2$, then the oscillation of $\text{Sin}[x^2]$ across $[-2, 1)$, then a straight segment of slope 2 rising to 6 at $x = 3$, then the line $y = x$ dropping to 3 and climbing again. The jumps at $x = -2$, $x = 1$ and $x = 3$ are genuine discontinuities; Plot draws them as steep connecting segments because it joins its sample points.